

A d a p t e r

Adapter Pattern

Make incompatible interfaces fit

DEFINITION

Convert a class's interface into another interface that clients expect.

Adapter wraps an existing class in the interface a client expects, translating calls so incompatible code works together unchanged. The Adapter implements the Target interface the client codes against and delegates to the Adaptee's mismatched API — the travel-plug pattern: convert one socket shape into another, nothing more.

WHY IT MATTERS

When you must reuse a class whose interface doesn't match — a third-party SDK, a legacy API — you often can't change the source, and editing every call site is worse. An adapter isolates the conversion in one place: the client depends only on your Target interface, and the Adaptee stays untouched. The cost is an extra layer that can mask a deeper model mismatch.

— How it works

Target	The interface the client codes against (request()).
Adaptee	The existing class with a useful but mismatched API.
Adapter	Implements Target, delegates to the Adaptee's methods.

HOW TO APPLY

1. Define the Target interface your client actually wants.
2. Wrap the Adaptee in an Adapter that implements Target.
3. Translate names, parameters, and types inside the adapter — keep it thin.
4. Depend on Target everywhere; the Adaptee stays behind the port.

LITMUS TEST

Do you have a useful class with the wrong interface that you can't (or shouldn't) change? If you need it to fit a contract it doesn't match, wrap it in an adapter.

— Smell → fix

The app is wired directly to a third-party SDK's shape:

```
// app code speaks Stripe's API everywhere
stripe.charges.create({ amount, source })
// swap to PayPal later → edit every call site
```

↓ WRAP IT BEHIND YOUR OWN PORT

```
interface PaymentGateway { charge(cents): Promise }
class StripeAdapter implements PaymentGateway {
  #sdk = new Stripe(key)
  charge(cents) {
    return this.#sdk.charges.create({ amount: cents })
  }
}
```

The app now depends on PaymentGateway, your interface; StripeAdapter translates to the SDK. Swapping to PayPal is a new adapter — no call site changes. Keep adapters thin: translate only, no business logic.

USE WHEN

- ✓ Integrate a third-party or legacy API
- ✓ Two interfaces almost match

AVOID WHEN

- ▲ You control both sides — fix the interface instead

— Trade-offs & pitfalls

PROS

+ Reuse without modifying either side

CONS

– An extra layer

– Can mask a deeper model mismatch

COMMON PITFALLS

- ▲ Don't adapt when you can change the source — fixing the interface directly is cleaner.
- ▲ Stacking adapters over adapters signals a missing abstraction, not a translation problem.
- ▲ Keep adapters thin: business logic belongs in the Adaptee or the domain, not the translator.

WHERE YOU SEE IT

- ▶ Wrapping axios / fetch or a payment SDK (Stripe, PayPal) behind your own port (hexagonal architecture).
- ▶ `Arrays.asList(arr)` adapting a raw array to `List`; `InputStreamReader` adapting bytes to a `char Reader`.
- ▶ ORM drivers and logger bridges presenting one interface over many backends.

SEE ALSO

Facade

also fronts other code — but invents a new simpler interface over many classes, not one

Decorator

same wrapping shape — but keeps the interface and adds behavior instead of converting it

Proxy

wraps one object too — but controls access rather than changing the interface

Bridge

looks similar but is planned up front to vary two sides, not retrofitted to fix a mismatch

— Interview & sources

Adapter vs Decorator?

Both wrap an object, but Adapter changes the interface to a different one; Decorator keeps the same interface and layers on behavior.

Object vs class adapter?

Object adapter composes the Adaptee (adapts its subclasses too); class adapter subclasses it and implements Target — fine in Java, impossible in JS. Prefer the object form.

Adapter vs Facade?

Adapter makes one existing class fit an expected interface; Facade invents a new, simpler interface over a whole subsystem.

Where do you meet it daily?

Wrapping third-party SDKs and legacy APIs behind your own port, so the app depends on your interface, not theirs.

REMEMBER

Wrap a class so its interface matches what the client expects — translate, don't rewrite.

SOURCES

GoF — "Design Patterns" (1994): Adapter (structural)

Freeman & Robson — "Head First Design Patterns" (2nd ed., 2020): Adapter & Facade